

# RAGNexus

Hybrid RAG Chatbot — Full Stack AI Application

Assignment 1 · Chatbot Development

Student Name

Roll Number

Course

Artificial Intelligence / NLP

Submission Date

FastAPI

React + Vite

Qdrant

Groq LLM

RAG Pipeline

PostgreSQL

## Table of Contents

---

|     |                                       |
|-----|---------------------------------------|
| 1.  | Project Overview & Chatbot Objective  |
| 2.  | Target Users & Use Case               |
| 3.  | System Architecture                   |
| 4.  | Conversation Flow Design              |
| 5.  | Chatbot Interactions (5 Demonstrated) |
| 6.  | Fallback & Error Handling             |
| 7.  | Tech Stack & Setup                    |
| 8.  | API Endpoints                         |
| 9.  | Screenshots & Outputs                 |
| 10. | Working Project Link                  |
| 11. | Conclusion                            |

## SECTION 1

# Project Overview & Chatbot Objective

---

RAGNexus is a full-stack, AI-powered chatbot built on a Retrieval-Augmented Generation (RAG) architecture. Unlike traditional chatbots that reply from a fixed script or a generic language model, RAGNexus grounds every answer in documents that the user explicitly ingests — URLs, PDFs, TXT files, or CSVs. This ensures responses are factual, traceable, and accompanied by source citations.

The system combines a FastAPI backend, a React + Vite frontend, a Qdrant vector database for semantic search, a BM25 keyword retriever, a cross-encoder reranker, and the Groq LLM (llama-3.1-8b-instant) for answer generation. When no document is loaded, the chatbot falls back to a general-purpose conversational mode.

### Core Objective

- Allow users to upload or link any document (PDF, URL, TXT, CSV)
- Answer natural-language questions strictly from ingested content
- Return answers with cited source chunks for full transparency
- Maintain persistent chat sessions and conversation history
- Provide a general fallback chat mode when no document is loaded

## SECTION 2

# Target Users & Use Case

---

RAGNexus is designed as a general-purpose intelligent document assistant. Its primary target audiences and their use cases are described below.

### Students & Researchers

Upload research papers, textbooks, or lecture notes and ask direct questions without reading the entire document. Ideal for summarising key findings or cross-referencing multiple sources.

### Customer Support Teams

Ingest product manuals, FAQs, or policy documents. The chatbot acts as a first-line support agent that retrieves precise answers from company documentation.

### Legal & Compliance Professionals

Upload contracts, regulations, or guidelines and query them for specific clauses, obligations, or definitions.

### Campus Assistants

Ingest handbooks, timetables, or notice boards and allow students to query course schedules, hostel rules, exam dates, and more.

SECTION 3

# System Architecture

RAGNexus is structured around two main data flows: the Document Ingestion Flow and the Question-Answering Flow.

## Document Ingestion Flow

- Source (URL / PDF / TXT / CSV)
- Loader (fetches & parses content)
- Chunker (splits into overlapping chunks)
- Embedder (sentence-transformers model)
- PostgreSQL (document & chunk metadata storage)
- Qdrant (dense vector storage)

## Question-Answering Flow

- User Question
- HyDE Expansion (generate hypothetical answer for better embedding)
- Hybrid Retriever (Qdrant dense search + BM25 keyword search)
- Cross-Encoder Reranker (re-scores retrieved chunks)
- Top-K Contexts selected
- Groq LLM (llama-3.1-8b-instant) generates grounded answer
- Answer + Source Citations returned to frontend

## Architecture Diagram

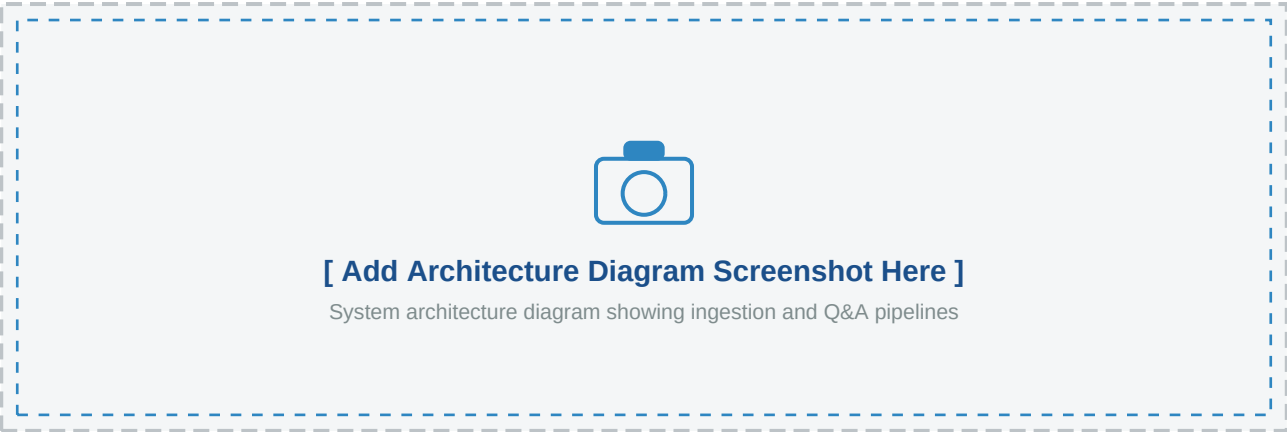


Figure 1 — RAGNexus System Architecture

SECTION 4

# Conversation Flow Design

The chatbot follows a structured conversation flow designed around two modes: Document Mode (RAG-grounded) and Basic Chat Mode (general LLM).

|   |                               |                            |
|---|-------------------------------|----------------------------|
| 1 | Welcome & Onboarding          | The frontend and creates   |
| 2 | Document Ingestion (optional) | The user pas success with  |
| 3 | User Query                    | The user typ along with th |
| 4 | Retrieval & Answer Generation | The backen returns a gro   |
| 5 | Follow-up & History           | The convers questions rel  |
| 6 | Fallback (No Document)        | If no docume /chat/basic w |

## Conversation Flow Diagram

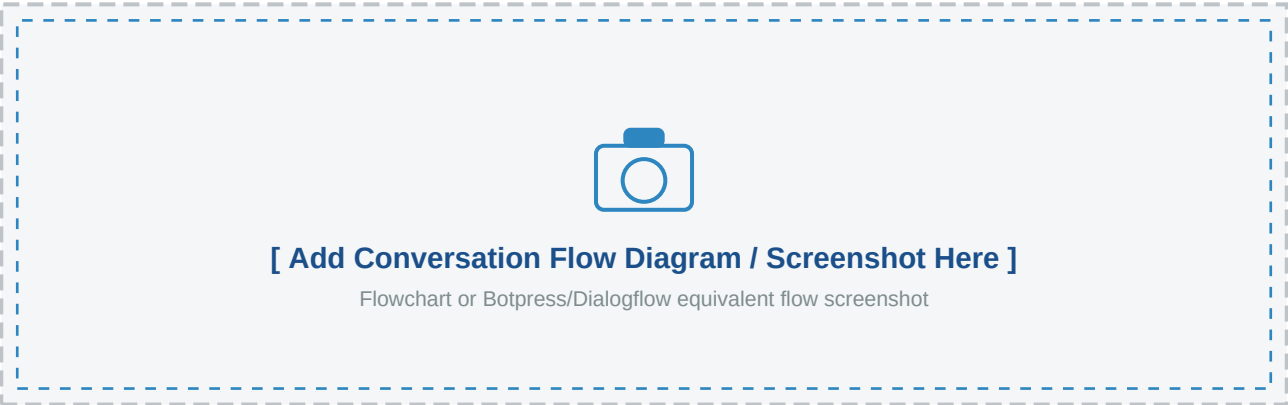


Figure 2 — Chatbot Conversation Flow

SECTION 5

Chatbot Interactions (5 Demonstrated)

The following five interactions demonstrate RAGNexus covering the full range of chatbot capabilities required by this assignment: document ingestion, targeted Q&A, follow-up conversation, source verification, and fallback handling.

| Interaction 1                                                                                                                                                                                                           |                                                                                                                                                     | Document |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Description:                                                                                                                                                                                                            | User submits a URL to ingest. The system loads, chunks, and embeds the content, then confirms success with a document_id.                           |          |
| User Input:                                                                                                                                                                                                             | <code>POST /ingest/url → { "url": "https://example.com/report.pdf" }</code>                                                                         |          |
| Bot Response:                                                                                                                                                                                                           | <code>{ "document_id": "d3f1a2...", "chunks": 42, "status": "ingested" }</code>                                                                     |          |
| <div><p>[ Add Screenshot of This Interaction ]</p><p>Screenshot showing Interaction 1: Document Ingestion via URL</p></div>           |                                                                                                                                                     |          |
| Interaction 2                                                                                                                                                                                                           |                                                                                                                                                     | Document |
| Description:                                                                                                                                                                                                            | User asks a factual question about the ingested document. RAGNexus retrieves relevant chunks and generates a grounded answer with source citations. |          |
| User Input:                                                                                                                                                                                                             | <code>"What are the key findings of this report?"</code>                                                                                            |          |
| Bot Response:                                                                                                                                                                                                           | <code>"The report identifies three key findings: (1)... (2)... (3)... [Source: chunk 7, 12]"</code>                                                 |          |
| <div><p>[ Add Screenshot of This Interaction ]</p><p>Screenshot showing Interaction 2: Document-Mode Question &amp; Answer</p></div> |                                                                                                                                                     |          |

## Interaction 3

Follow-Up

**Description:** User asks a follow-up question referencing the previous answer. The system uses session history to provide a contextually coherent response.

**User Input:** "Can you elaborate on finding number 2?"

**Bot Response:** "Finding 2 refers to... based on section 3 of the document. [Source: chunk 14]"



[ Add Screenshot of This Interaction ]

Screenshot showing Interaction 3: Follow-Up Conversation (Session History)

## Interaction 4

Source C

**Description:** User requests to see the raw source chunks used to generate the last answer, demonstrating full transparency of the RAG pipeline.

**User Input:** "Show me the sources used for that answer."

**Bot Response:** "Sources: [Chunk 7] ... [Chunk 12] ... [Chunk 14] — retrieved from document d3f1a2"



[ Add Screenshot of This Interaction ]

Screenshot showing Interaction 4: Source Citation Verification



| Interaction 5                                                                                                                                                                                                       |                                                                                                                                                       | Basic Chat |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Description:                                                                                                                                                                                                        | User sends a general question outside any document context. The system routes to basic chat mode using Groq directly, with no retrieval constraints.  |            |
| User Input:                                                                                                                                                                                                         | <i>"What is retrieval-augmented generation?"</i>                                                                                                      |            |
| Bot Response:                                                                                                                                                                                                       | <i>"RAG is an AI technique that combines a retrieval system with a language model to produce grounded answers from external knowledge sources..."</i> |            |
| <div><p>[ Add Screenshot of This Interaction ]</p><p>Screenshot showing Interaction 5: Basic Chat Fallback (No Document)</p></div> |                                                                                                                                                       |            |

SECTION 6

# Fallback & Error Handling

RAGNexus implements multi-level fallback and error handling to ensure the chatbot always responds gracefully regardless of user input.

 **No Document Loaded**

 **Context Not Found in Document**

 **Invalid File Type**

 **Qdrant / Database Unavailable**

 **Malformed / Empty Input**



**[ Add Fallback Response Screenshot Here ]**

Show a query that triggers the fallback/error response

Figure 3 — Fallback Response in Action

## SECTION 7

## Tech Stack & Setup

|                   |                                          |
|-------------------|------------------------------------------|
| Backend Framework | FastAPI + SQLAlchemy (Python 3.10+)      |
| Frontend          | React + Vite (Node.js 18+)               |
| Vector Database   | Qdrant (Docker)                          |
| Relational DB     | PostgreSQL / Supabase                    |
| LLM Provider      | Groq (llama-3.1-8b-instant)              |
| Embeddings        | sentence-transformers (all-MiniLM-L6-v2) |
| Keyword Retrieval | BM25 (rank_bm25)                         |
| Reranker          | Cross-Encoder (sentence-transformers)    |
| Evaluation        | RAGAs (optional)                         |

### Quick Setup Steps

1. Clone repository and create Python virtual environment
2. `pip install -r requirements.txt`
3. `cd web && npm install`
4. Create `.env` with `GROQ_API_KEY`, `DATABASE_URL`, `QDRANT_URL`
5. `docker run -p 6333:6333 qdrant/qdrant`
6. `uvicorn app.main:app --reload --port 8000`
7. `cd web && npm run dev` → open `http://localhost:5173`

## SECTION 8

## API Endpoints

---

RAGNexus exposes a clean REST API. Below is a summary of all available endpoints.

| Method | Endpoint                   | Description                                 |
|--------|----------------------------|---------------------------------------------|
| GET    | /health                    | Health check — confirms backend is running  |
| POST   | /ingest/url                | Ingest content from a URL                   |
| POST   | /ingest/file               | Ingest PDF, TXT, or CSV file upload         |
| POST   | /ask                       | Document-mode Q&A; (requires document_id)   |
| POST   | /chat/basic                | General chat without document retrieval     |
| POST   | /chat/sessions             | Create a new chat session                   |
| GET    | /chat/sessions             | List all chat sessions for current user     |
| GET    | /chat/history/{session_id} | Retrieve full message history for a session |

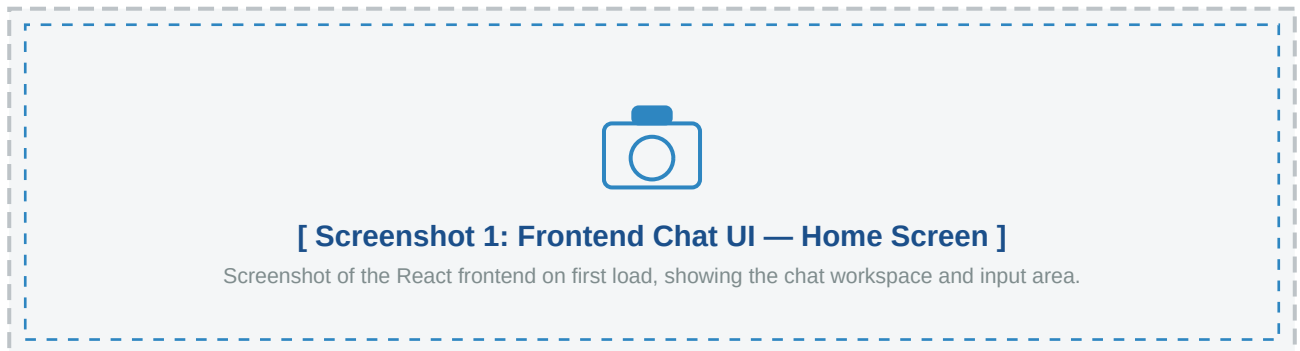
## SECTION 9

## Screenshots & Outputs

Add screenshots of the chatbot UI and key interactions below. Each placeholder indicates exactly what screenshot to insert.

### Screenshot 1: Frontend Chat UI — Home Screen

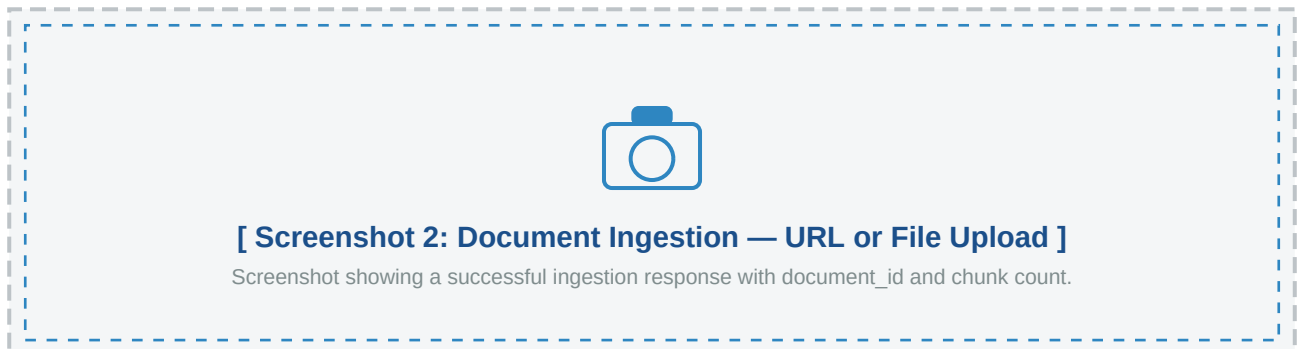
*Screenshot of the React frontend on first load, showing the chat workspace and input area.*



*Figure 4 — Frontend Chat UI — Home Screen*

### Screenshot 2: Document Ingestion — URL or File Upload

*Screenshot showing a successful ingestion response with document\_id and chunk count.*



*Figure 5 — Document Ingestion — URL or File Upload*

### Screenshot 3: Document Q&A; — Answer with Source Citations

*Screenshot of a user question and the chatbot answer with highlighted source chunks.*



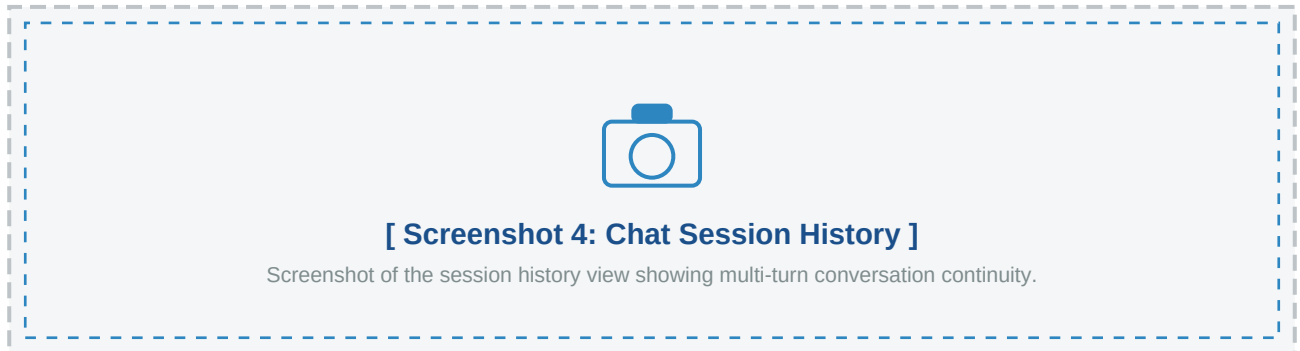
**[ Screenshot 3: Document Q&A — Answer with Source Citations ]**

Screenshot of a user question and the chatbot answer with highlighted source chunks.

*Figure 6 — Document Q&A; — Answer with Source Citations*

## Screenshot 4: Chat Session History

*Screenshot of the session history view showing multi-turn conversation continuity.*



*Figure 7 — Chat Session History*

## Screenshot 5: Basic Chat Mode (Fallback)

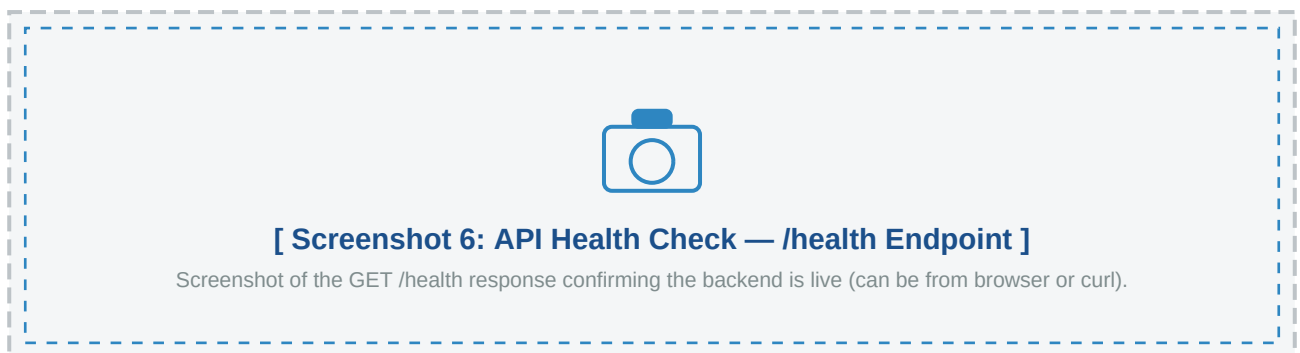
*Screenshot of a general question answered in basic chat mode without document context.*



*Figure 8 — Basic Chat Mode (Fallback)*

## Screenshot 6: API Health Check — /health Endpoint

*Screenshot of the GET /health response confirming the backend is live (can be from browser or curl).*



*Figure 9 — API Health Check — /health Endpoint*

## SECTION 10

## Working Project Link

Provide the live deployment URL or a recorded demo link below. If running locally, a screen-recorded walkthrough video is acceptable.

|                      |                      |
|----------------------|----------------------|
| Live Frontend URL    | <input type="text"/> |
| Backend API Base URL | <input type="text"/> |
| GitHub Repository    | <input type="text"/> |
| Demo Video Link      | <input type="text"/> |



**[ Add Deployment / Live Demo Screenshot Here ]**

Show the app running on the deployed URL or localhost

Figure 10 — Live Deployment or Local Demo



SECTION 11

# Conclusion

RAGNexus demonstrates a production-quality chatbot that satisfies all requirements of Assignment 1 and goes significantly beyond a basic implementation.

|   |                              |                                |
|---|------------------------------|--------------------------------|
| ✓ | Clear Objective & Use Case   | RAGNexus s                     |
| ✓ | Conversation Flow Design     | A structured<br>implemented    |
| ✓ | Built on a Suitable Platform | Full-stack ap                  |
| ✓ | 5 Meaningful Interactions    | URL ingestio<br>demonstrated   |
| ✓ | Fallback / Error Handling    | Multi-level fa<br>database una |

Beyond the assignment requirements, RAGNexus showcases advanced NLP techniques including Hypothetical Document Embeddings (HyDE), hybrid dense+sparse retrieval, cross-encoder reranking, and optional RAGAs evaluation — reflecting real-world production RAG system design.